

```
user@user:~$
```

Introduction to Bash

Document compiled, owned and managed by: [Shashank Khobragade](#)
E-Mail: shashank_dk@hotmail.com

Table of contents

[What is Bash?](#)

[What is the connection between Bash and Linux, and what is a shell?](#)

[The local and the cloud terminals](#)

[Basic Bash Commands](#)

[File paths: Relative and Absolute](#)

[Commands for files and directories](#)

[Commands for copy, move and rename](#)

[Finding files/directories](#)

[Combining commands with ;](#)

[Piping with |](#)

[Streaming output to a file](#)

[Reading input from the command line](#)

[Regex with grep](#)

[Variables in Bash](#)

[Text editors](#)

[Bash Scripting](#)

[Customizing the terminal prompt](#)

What is Bash?

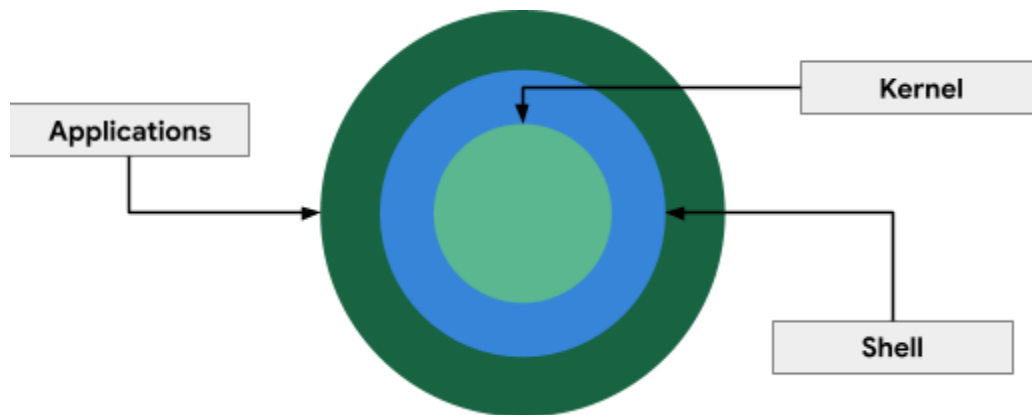
Bash is the default shell on almost all the Linux distributions. Although there are other shells available such as zsh, ksh, fish, etc., bash is the most popular shell and runs on almost all Unix and Unix-like operating systems. It derives its name from **B**ourne-**A**gain **S**hell named after Stephen Bourne.

What is the connection between Bash and Linux, and what is a shell?

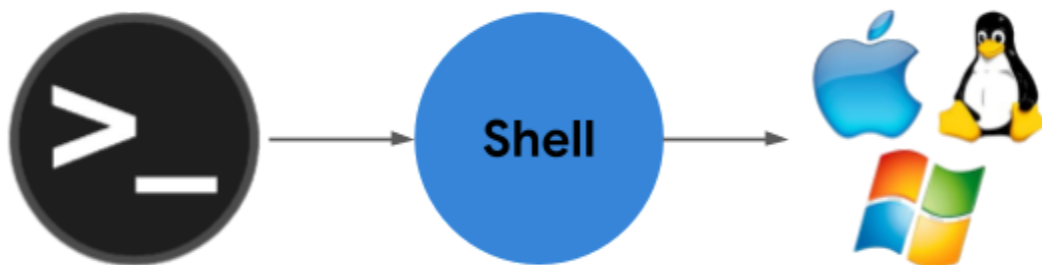
Linux is a Unix-like operating system, although to be precise it is the Kernel that forms the core of an operating system. The Linux distribution on the other hand is the combination of Linux kernel and GNU apps and tools, a package management system along with documentation.

See: [Operating Systems: Timeline and Family Tree](#)

A shell is the outer layer of the kernel that enables the user to interact with it with the help of commands. The outermost layer consists of the GNU applications which provide GUI tools.



The operating system receives these commands from the shell and performs the requested actions. To run these commands on a shell, however, they need to be written in a terminal.



```
user@user:~$
```

Bash Commands

Basic Bash Commands

```
user@user:~$ sudo apt install <command>
```

Installs the **command**

```
sudo apt install
```

Installs a command to be used. If a command prints an error message: **Command not found**, using `sudo apt install` will install it.

```
user@user:~$ echo
```

Prints content to the terminal

```
echo "Hello, world"  
echo Ubuntu  
echo 'This is a string'
```

Prints the content followed by **echo** on the terminal. It can be also used to print the content stored inside a variable.

```
user@user:~$ pwd
```

Prints the name of the current/working directory

```
pwd
```

Prints the name of the current/working directory on the terminal

```
user@user:~$ man <command>
```

Opens the interface with the system reference manual

```
man echo  
man pwd  
man man  
man ls  
man bash
```

Opens the interface with the system reference manual for a command which contains the description of the command, including its syntax and information about the various flags that can be used with it.

user@user:~\$ help <command> Display information about builtin commands	
help pwd	Help prints a short summary of the builtin commands on the terminal itself
user@user:~\$ clear Clears the terminal screen	
clear	Clears the terminal screen
clear -x	Clears the terminal screen, but the previous screen can be accessed by scrolling up. Alternatively ctrl L can be used.
user@user:~\$ date Prints the date	
date	Prints the current date on the terminal in the format: Wed Jul 2 10:30:05 AM UTC 2024 The date can be formatted by using flags.
date --date="2 days ago"	Prints the date from 2 days ago
date -d="2 days"	Prints the date after 2 days
date --d="1 year ago"	Prints the date from 1 year ago
date +%T	Prints the time in hh:mm:ss format
date +%F	Prints the full date in the format yyyy-mm-dd
date +"%T %d.%m.%y"	Prints the date in the format HH:MM:SS dd.mm.yyyy
user@user:~\$ ncal user@user:~\$ cal Prints the calendar	
ncal	Prints the current calendar month vertically highlighting today
cal	Prints the current calendar month horizontally

<pre>ncal -y ncal -y 1991</pre>	Prints the calendar for current year Prints the calendar for the year 1991 (When Linux Torwald started creating Linux kernel)
<pre>ncal -m 12 ncal -m 12 2023</pre>	Prints the calendar for December Prints the calendar for December 2023
<pre>ncal -h -j -M -3 ncal -hjM3</pre>	Prints the calendar without highlighting the current day + the day numbers + week starting on Monday + previous, current, next month
<pre>user@user:~\$ type <command></pre> Prints the type of the command	
<pre>type <command></pre>	Prints the type of the command: shell builtin, executable, etc.
<pre>type echo</pre>	Prints: echo is a shell builtin
<pre>type nlp</pre>	Prints: nlp is hashed (/usr/bin/nlp) which is the location of the command i.e. it's located in the /usr/bin/ directory
<pre>user@user:~\$ which <command></pre> Prints the absolute path for the command	
<pre>which <command></pre>	Prints the location (absolute path) of the command

File paths: Relative and Absolute	
<pre>user@user:~\$</pre>	
Relative path	Relative file path refers to a file/directory location that is relative to the current location . notation refers to the current directory .. notation refers to the parent directory relative to the current directory and can be used to move up one level in the hierarchy

<pre>user@user:~/Desktop/Images\$</pre>	The <i>Images</i> directory is the direct child directory relative to <i>Desktop</i>, whereas <i>Desktop</i> is the parent directory relative to <i>Images</i> . refers to the <i>Images</i> directory .. refers to the <i>Desktop</i> directory
<pre>user@user:~/Desktop/Videos\$</pre>	To browse the content of <i>Videos</i> while you're located inside <i>Images</i>, the relative path will be ../Videos
Absolute path <pre>user@user:~/Desktop/Videos\$ pwd</pre> <pre>user@user:~/Desktop/Videos\$ which echo</pre> <pre>/usr/bin/echo</pre> ~ /	Absolute path refers to the fixed address for a directory or a file location and can be accessed from anywhere. It always begins with the topmost level in the hierarchy i.e. the root directory in Linux systems denoted by / The commands pwd and which will always print the absolute path on the terminal screen. pwd prints the absolute path for the <i>Desktop</i> directory located on the local terminal. The command echo has the absolute path /usr/bin/echo The tilde ~ denotes the absolute path for the home directory The forward slash / denotes the absolute path for the root directory that contains all the files and directories within the operating system

Commands for files and directories	
<pre>user@user:~\$ ls</pre> Prints a list of the files and directories inside a directory	
ls	Prints the list of all the files and folders in the

	current working directory
ls -l	Prints the list of all the files and folders in the current working directory in the long format i.e. with additional details about each of them
ls -a	Prints the list of all the files and folders in the current working directory including the hidden ones starting with ., and the directory references . (for current directory) and .. (directory one level up)
ls -l -a ls -la	Prints the list of all the files and folders including the hidden ones and the references . and .. in the long format
ls ~	Prints the list of files and folders in the home directory ~ is used as a reference for home directory
ls /	Prints the list of files and folders in the root directory / is used as a reference for root directory
ls ..	Prints the list of files and folders in the parent directory of the current directory
ls ~/Desktop	Prints the list of files and folders using the absolute path for the <i>Desktop</i> directory
user@user:~\$ mkdir <dir>	
Creates a new directory	
mkdir Google	Makes a directory named Google
mkdir Microsoft Linux macOS	Makes three directories Microsoft Linux and macOS
mkdir "My Files"	Makes a directory called My Files
mkdir "My Files/My Work Files"	Makes a nested directory My Work Files inside My

mkdir -P Images/jpeg Images/png	Files Creates a nested directory structure with Image as parent directory (-P) and its two sub-directories jpeg and png
mkdir ~/Super_dir	Makes a directory Super_dir in the home directory
user@user:~\$ rmdir <dir> Removes an existing directory	
rmdir Google	Deletes the empty directory Google
rmdir Microsoft Linux macOS	Deletes the three empty directories Microsoft Linux and macOS
user@user:~\$ cd <dir/path> Changes the working directory	
cd	Change (working) directory switches to the mentioned directory (name/path)
cd Downloads	Changes working directory to Downloads
cd Downloads/Docs	Changes the working directory to Docs inside the Downloads directory (uses relative path)
cd .	Changes the working directory to the current directory (Nothing will change on the screen)
cd ..	Changes the working directory to the parent directory
cd ~ cd	Changes the working directory to home
cd /	Changes the working directory to the root
cd ~/Downloads/Images	Changes the working directory to Images using the absolute path i.e. Images inside Downloads inside Home

user@user:~\$ touch <file_name>

Creates a new file

touch my_text_file.txt

Creates the file **my_text_file.txt** in the current directory (file extensions are optional, but recommended to specify the type of the file)

touch file_1.py file_2.sh file_3.txt

Creates three files in the current directory

touch ~/in_home.txt

Creates the file **in_home.txt** in the home directory

touch Documents/README.md

Creates the file **README.md** in the *Documents* directory (using relative path)

user@user:~\$ file <file_name>

Prints the type of the file

file diff_calculator.py

Prints what type of file it is

user@user:~\$ rm <file_name>

Deletes the file

rm my_text_file.txt

Removes/Deletes the file **my_text_file.txt**

rm -d My_Folder

Deletes the empty directory **My_Folder** (alternative for *rmdir*)

rm -r My_Folder

Deletes the non-empty directory **My_Folder** and all its contents recursively

rm -i python_script.py bash_script.sh

Interactively prompts for confirmation before deleting the files

rm -dri Documents

Interactively prompts for confirmation before deleting the **Documents** directory and all its content recursively

user@user:~\$ cat <file_name>

Print the content of the file/s by concatenating

cat error.log	Prints (concatenates) the content of the file error.log
cat output.txt error.log	Concatenates the content of the files and prints them in the order
cat -n song.txt cat --number song.txt	Prints the content of song.txt with line numbers
user@user:~\$ less <file_name> Display content of the file in scrollable manner	
less very_long_file.txt	Prints the content of very_long_file.txt in a scrollable interface. Search options can also be used in the interface.
less -N very_long_file.txt less --LINE-NUMBERS very_long_file.txt	Prints the content of very_long_file.txt in a scrollable interface with line numbers
user@user:~\$ head <file_name> Print the first ten lines of a file	
head error.log	Prints the first 10 lines of error.log
head -n 50 ldaps.txt head -50 ldaps.txt	Prints the first 50 lines in the ldaps.txt file
user@user:~\$ tail <file_name> Print the last ten lines of a file	
tail error.log	Prints the last 10 lines of error.log
tail -n 50 ldaps.txt tail -50 ldaps.txt	Prints the last 50 lines in the ldaps.txt file
user@user:~\$ sort <file_name> Sort the file content	
sort examples.txt	Sorts the content of examples.txt in alphabetical order

sort -n expenses.txt	Sorts the content of expenses.txt numerically
sort -r examples.txt	Sorts the content of examples.txt in alphabetically in reverse order
sort -u duplicates.txt	Sorts the content of duplicates.txt uniquely (i.e. by omitting the duplicates)
sort -k 2 names.txt	Sorts the contents of names.txt according to the column key 2 i.e. 2nd column
user@user:~\$ wc <file_name> Get the word count of the file	
wc don_quixote.txt	Prints the number of lines, words and characters/bytes in the file don_quixote.txt
wc -l don_quixote.txt	Prints the number of lines in the file don_quixote.txt
wc -w don_quixote.txt	Prints the number of words in the file don_quixote.txt
wc -c don_quixote.txt	Prints the number of characters in the file don_quixote.txt
wc -m don_quixote.txt	Prints the number of bytes in the file don_quixote.txt
user@user:~\$ tac <file_name> Reverse cat to print content in vertically reverse order	
tac timestamp.log	Prints the content of timestamp.txt in reverse order vertically (reverse cat → tac)
user@user:~\$ rev <file_name> Prints the content line by line horizontally reverse order	
rev palindrome.txt	Prints the content of palindrome.txt line by line in horizontally reverse order

Commands for copy, move and rename

user@user:~\$ cp <source> <destination>

Copy source(s) file(s)/directory(s) to destination

cp block_n_ground.txt block_n_floor2.txt

Copies the content of the file **block_n_ground.txt** to **block_n_floor2.txt** (block_n_floor2.txt will be created if it doesn't already exist)

cp -i block_n_ground.txt block_n_floor2.txt

Copies the content of the file **block_n_ground.txt** to **block_n_floor2.txt** interactively by prompting for confirmation before overwriting

cp Docs file.txt source.log Destination

Copies multiple sources - **Docs** directory, **file.txt** and **source.log** files in the **Destination** directory

*There can be multiple sources but the last argument will be the destination folder, it cannot be a file

user@user:~\$ mv <source> <destination>

Rename source file/directory as destination file/directory, or move source(s) to destination folder

mv nlp.txt gemini.txt

Renames **nlp.txt** to **gemini.txt** (if the latter doesn't exist) or moves the content of **nlp.txt** to **gemini.txt** (if the latter already exists)

mv nlp.txt Gemini

Moves the file **nlp.txt** to the existing **Gemini** folder

Finding files/directories

user@user:~\$ locate <file/directory>

Quickly finds the file/directory

locate python

Finds all the files/folders on the system containing the word **python** anywhere in the file path

locate -c python	Finds all the file/folders on the system containing the word python and outputs the number of occurrences
locate -b python	Finds all the files/folders on the system containing the word python only in the base name (See the count difference using locate -c python and locate -bc python)
user@user:~\$ find <file/directory> Quickly finds the file/directory	
find	Displays the entire file-folder structure of the working directory including the hidden files and nested files/folders
find -type d	Finds only directories
find -type f	Finds only files
find *.txt	Finds files with .txt extension (* matches with any number of characters before .txt)
find -name "*123*"	Finds all the files and folders with the pattern *123* i.e. containing 123 in it
find -empty	Finds all the empty files and folders in the working directory

Combining commands with ;	
user@user:~\$ command_1; command_2; ...; command_n The commands run in the chronological order after the previous one is executed	
mkdir pygmalion_dublin; cd pygmalion_dublin; touch de.txt es.txt fr.txt it.txt; ls -l	Creates the directory pygmalion_dublin → changes the working directory to pygmalion_dublin → creates four .txt files → displays the list of files and folders inside pygmalion_dublin

mkdir dir{1..10}; ls -l; rmdir dir{1..5}; ls -l	Creates 10 directories dir1 , dir2 , ..., dir10 → lists the files and folder in the working directory → deletes dir1 , dir2 , ..., dir5 → lists the files and folders in the working directory
head countries.txt; tail countries.txt	<i>head</i> will display the first 10 lines in countries.txt followed by the last 10 lines showed by <i>tail</i>

Piping with |

```
user@user:~$ command_1 | command_2 | ... | command_n
```

The commands run in the chronological order. The output of the previous command is accepted as the input by the next command. Piping works only for commands that provide output (e.g. echo, cat, ls) followed by one that requires an argument as input (ls, cat)

head -30 countries.txt tail -10	head -30 outputs the first 30 lines → tail -10 takes these lines as input → tail -10 outputs the last 10 lines from this input
man echo head -20	man echo displays the user manual for echo → head -20 accepts it as input → head -20 displays the first 20 lines of man echo
history tail -20	history displays the list of commands performed in the session → tail takes it as input and displays the last 20 commands

Streaming output to a file (stdout: Standard output)

```
user@user:~$ command > file_path
user@user:~$ command >> file_path
```

The output produced by a command can be redirected to a text file. It can either be overwritten using > or appended in a new line using >>

echo "Hello, world!" > file.txt	echo outputs <i>Hello, world!</i> → > redirects it to file.txt * → <i>Hello, world</i> is (over)written in file.txt Use <i>cat file.txt</i> to see the content.
echo "Hello, world" >> file.txt	echo outputs <i>Hello, world!</i> → >> redirects it to

<pre>date > timestamp.log</pre> <pre>date >> timestamp.log</pre>	file.txt* → <i>Hello, world</i> is appended in file.txt on a new line Use <i>cat file.txt</i> to see the content. date outputs the current datetime → written in the file timestamp.log* Rerunning the command will overwrite the datetime date outputs the current datetime → appended in the file timestamp.log* Rerunning the command will append the datetime on a new line each time *If the file doesn't already exist in the working directory, it will be created. The content of the file can be seen either in the text editor, or using cat.
--	--

Reading input from the command line	
<pre>user@user:~\$ read</pre> <pre>user@user:~\$ read VAR</pre> Reads the input from the command line, and also stores it in a variable	
<pre>read VAR</pre> <pre>echo \$VAR</pre> <pre>read -p "Enter name: " NAME</pre> <pre>echo \$NAME</pre>	read awaits an input in the terminal → saves the input in VAR → echo prints the value inside VAR read prompts the user to <i>Enter name:</i> → saves the input in NAME → echo prints the value inside NAME

Regex with grep	
<pre>user@user:~\$ grep 'pattern' <file_name></pre> <pre>user@user:~\$ <command> grep 'pattern'</pre> Uses basic Regex to match the pattern in a given file line by line, or in the stdout (standard output) from another command	
<pre>grep '^S.*a\$' countries.txt</pre>	Displays a list of countries starting with S and

grep 'e{2}' countries.txt	ending with a in the file countries.txt
grep 'e\{2\}' countries.txt	Displays nothing because ? + { } () need to be escaped with <i>basic grep</i>
grep -P 'e{2}' countries.txt	Displays a list of countries with two consecutive e 's in it ({ } needed to be escaped)
echo "USER ID: abc1234567890" grep -P "abc\d+"	-P activates the Perl Compatible Regex (PCRE) and hence { } need not be escaped
echo "USER ID: abc1234567890" grep -oP "abc\d+"	echo outputs USER ID: nlp1234567890 → grep matches the pattern and prints USER ID: <u>abc1234567890</u> (The entire line is printed with the pattern in red)
grep -c '^A.*a\$' countries.txt	echo outputs USER ID: nlp1234567890 → grep -oP matches the pattern and prints abc1234567890 -o prints only the match and not the entire line
grep -i '^a.*a\$' countries.txt	Counts the number of matches for the pattern
<pre>user@user:~\$ egrep 'pattern' <file_name></pre> <pre>user@user:~\$ <command> egrep 'pattern'</pre> egrep stands for extended grep	
egrep 'e{2}' countries.txt	egrep solves the issue with escaping { } and }

Variables in Bash

```
user@user:~$ VAR=value
```

```
user@user:~$ echo $VAR
```

Variables in Bash can be used like other languages to store values, as well as to store output from a command

Rules for variables:

1. Variable name may contain letters, numbers and underscores

2. No whitespace is allowed before or after the assignment operator =
3. Variable name cannot start with a number
4. Variable name cannot contain special characters
5. Variable name cannot be same as a reserved keywords
6. No whitespaces are allowed in the variable name
7. The values stored in a variable can be retrieved using \$ sign before the variable name

<pre>VAR="Hello, world!" VAR123=42 _Variable="I am a variable" _variable_=Bash TRUE_VAR=true echo \$VAR echo \$VAR123 echo \$TRUE_VAR echo "The five variables are \$VAR, \$VAR123, \$_Variable, \$_variable_ and \$TRUE_VAR"</pre>	Valid variable names and their respective values Prints the corresponding variable value Prints the variable values in the provided string format
user@user:~\$ VAR=\$(command) The output of a command can be stored inside a variable using the command expansion syntax \$()	
<pre>Colours=\$(cat colours.txt); echo \$Colours first=\$(head countries.txt) last=\$(tail countries.txt) echo "\$first \$last"</pre>	The output from cat colours.txt is stored in the variable Colours and echo prints its value first contains the first 10 lines in countries.txt last contains the last 10 lines in countries.txt echo prints the two variable values

Text editors

```
user@user:~$ nano <file_name>
user@user:~$ vim <file_name>
user@user:~$ emacs <file_name>
```

Linux is bundled with some tools including the text editors **nano**, **vim** and **emacs** to write different types of files such as text files, log files, scripts for multiple programming languages,

etc.	
nano text_file.txt	The file text_file.txt is opened in the Nano Text Editor. If the file doesn't already exist, then it is created in the working directory. Ctrl O or Ctrl S to save the file Ctrl X to exit the editor
vim text_file.txt	The file text_file.txt is opened in the Vim Text Editor. If the file doesn't already exist, then it is created in the working directory. Type :wq and press Enter to exit
emacs text_file.txt	The file text_file.txt is opened in the Emacs Text Editor. If the file doesn't already exist, then it is created in the working directory.
nano vim emacs	Running only the editor name as commands will open the editor

Bash Scripting
<pre>user@user:~\$ touch my_first_bash_script.sh user@user:~\$ bash my_first_bash_script.sh</pre> A bash script may be stored as a file with the extension .sh. The command to run the script is bash followed by the file name/path. Write and save the following bash scripts using any of the text editors and run them using the format bash <file_name>.sh <u>Bash script 1: my_first_bash.sh</u> <pre>echo "Enter your name: " read NAME echo "Welcome to Bash Programming, \$NAME"</pre> <u>Bash script 2: countries.sh</u> <pre>PATTERN="^A.*a\$"</pre>

```
cat countries.txt |  
grep -oP $PATTERN > countries_filtered.txt  
cat countries_filtered.txt
```

```
user@user:~$
```

Extras

Customizing the terminal prompt

```
MY_CUSTOM_PROMPT $ nano ~/.bashrc
```

The terminal prompt can be customized by modifying the values of the **PS1** variable in the if-then loop inside the file called **.bashrc** located in the home directory.

- Create your custom prompt on [Easy Bash Prompt Generator](#)
- Copy **PS1=<prompt>**
- Open **.bashrc** in nano: `nano ~/.bashrc`
- Comment out the existing PS1 variables by writing **#** before it and paste the new PS1 variable
- Ctrl O or Ctrl S to save the .bashrc file and Ctrl X to exit nano
- Implement the changes: `source ~/.bashrc`